PIPELINED RISC CPU

SAHEEL FAISAL

PIPELINED RISC CPU

10-34

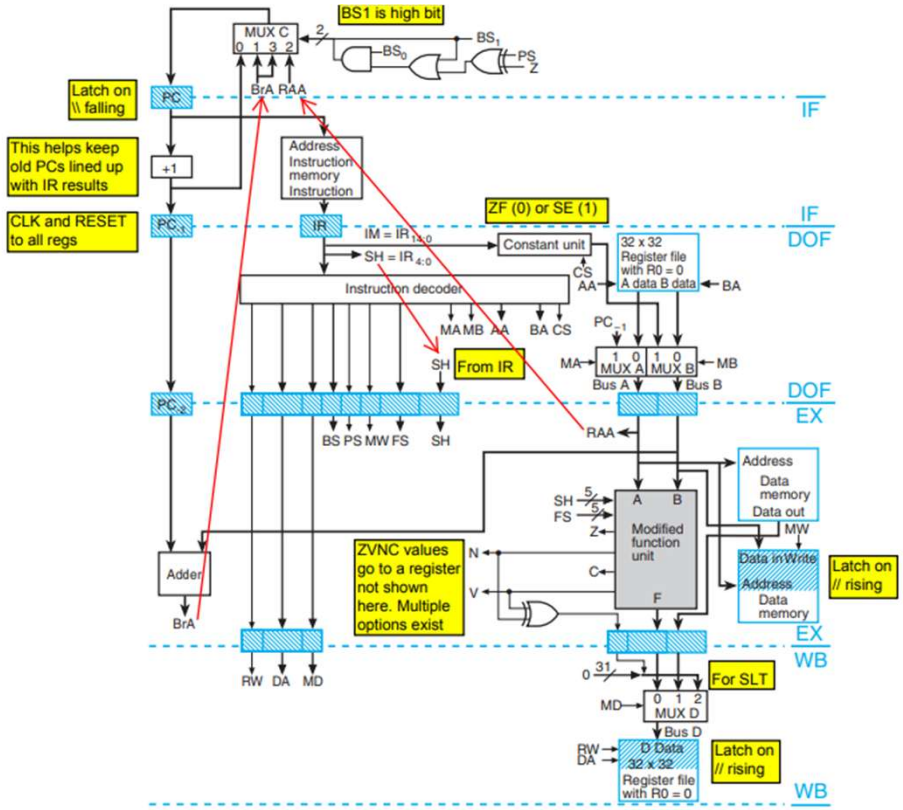


Fig. 10-15 Pipelined RISC CPU

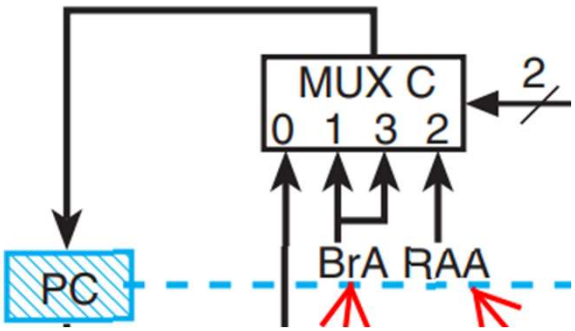
© 2001 Prentice Hall, Inc.
M. Morris Mano & Charles R. Kime
LOGIC AND COMPUTER DESIGN FUNDAMENTALS. 2e. Updated.

Divided into modules:

- Mux C
- Constant Unit
- Instruction Decoder
- Register File
- Mux A & B
- Function Unit
- Data Memory
- Mux D
- Instruction Memory
- RISC_CPU (Top module)

Mux C

```
module mux_C (
    input [1:0] sel,
    input [31:0] PC_1, BrA, RAA,
    output reg [31:0] PC
);
    always @(*) begin
        case (sel)
            2'b00: PC = PC_1;
            2'b01: PC = BrA;
            2'b10: PC = RAA;
            2'b11: PC = BrA;
            default: PC <= PC_1;
        endcase
    end
endmodule
```

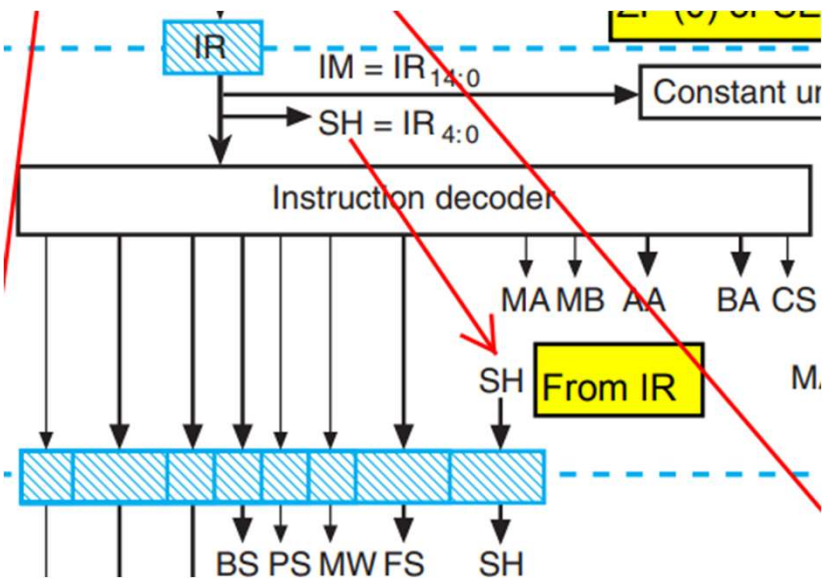


Constant Unit

constant_unit.v

```
module constant_unit (
    input CS,
    input [14:0] IM,
    output [31:0] IM_extended
);

    assign IM_extended = CS ? {{17{IM[14]}}, IM}: {17'b0, IM};

endmodule
```

Instruction Decoder




```
module instruction_decoder (
    input [31:0] IR,
    output RW, MW, MB, MA, CS, PS,
    output [1:0] MD, BS,
    output [4:0] FS, AA, BA, DA
);

    assign DA = IR[24:20];
    assign AA = IR[19:15];
    assign BA = IR[14:10];

    wire [6:0] opcode;
    assign opcode = IR[31:25];
    reg [14:0] control_word;

    assign RW  = control_word[14];
    assign MD  = control_word[13:12];
    assign BS =  control_word[11:10];
    assign PS =  control_word[9];
    assign MW =  control_word[8];
    assign FS =  control_word[7:3];
    assign MB  = control_word[2];
    assign MA  = control_word[1];
    assign CS  = control_word[0];
```

Instruction Decoder

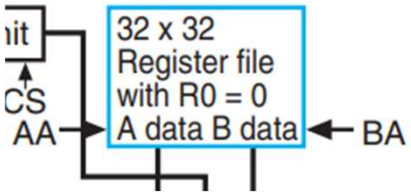
TABLE 10-20
Control Words for Instructions

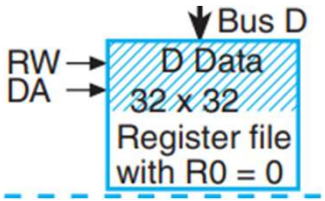
Symbolic Notation	Action	Op Code	RW	MD	BS	PS	MW	FS	MB	MA	CS	
			Control Word Values									
NOP	None	0000000	0	—	00	—	0	—	—	—	—	
ADD	$R[DR] \leftarrow R[SA] + R[SB]$	0000010	1	00	00	—	0	00010	0	0	—	
SUB	$R[DR] \leftarrow R[SA] + \overline{R[SB]} + 1$	0000101	1	00	00	—	0	00101	0	0	—	
SLT	If $R[SA] < R[SB]$ then $R[DR] = 1$	1100101	1	10	00	—	0	00101	0	0	—	N XOR V
AND	$R[DR] \leftarrow R[SA] \wedge R[SB]$	0001000	1	00	00	—	0	01000	0	0	—	
OR	$R[DR] \leftarrow R[SA] \vee R[SB]$	0001010	1	00	00	—	0	01010	0	0	—	
XOR	$R[DR] \leftarrow R[SA] \oplus R[SB]$	0001100	1	00	00	—	0	01100	0	0	—	
ST	$M[R[SA]] \leftarrow R[SB]$	0000001	0	00	00	—	1	—	0	0	—	
LD	$R[DR] \leftarrow M[R[SA]]$	0100001	1	01	00	—	0	—	—	0	—	
ADI	$R[DR] \leftarrow R[SA] + \text{se } IM$	0100010	1	00	00	—	0	00010	1	0	1	
SBI	$R[DR] \leftarrow R[SA] + \overline{(\text{se } IM)} + 1$	0100101	1	00	00	—	0	00101	1	0	1	
NOT	$R[DR] \leftarrow \overline{R[SA]}$	0101110	1	00	00	—	0	01110	—	0	—	
ANI	$R[DR] \leftarrow R[SA] \wedge (0 \parallel IM)$	0101000	1	00	00	—	0	01000	1	0	0	
ORI	$R[DR] \leftarrow R[SA] \vee (0 \parallel IM)$	0101010	1	00	00	—	0	01010	1	0	0	
XRI	$R[DR] \leftarrow R[SA] \oplus (0 \parallel IM)$	0101100	1	00	00	—	0	01100	1	0	0	
AIU	$R[DR] \leftarrow R[SA] + (0 \parallel IM)$	1100010	1	00	00	—	0	00010	1	0	0	
SIU	$R[DR] \leftarrow R[SA] + \overline{(0 \parallel IM)} + 1$	1000101	1	00	00	—	0	00101	1	0	0	
MOV	$R[DR] \leftarrow R[SA]$	1000000	1	00	00	—	0	00000	—	0	—	
LSL	$R[DR] \leftarrow \text{lsl } R[SA] \text{ by } SH$	0110000	1	00	00	—	0	10000	—	0	—	
LSR	$R[DR] \leftarrow \text{lsr } R[SA] \text{ by } SH$	0110001	1	00	00	—	0	10001	—	0	—	
JMR	$PC \leftarrow R[SA]$	1100001	0	—	10	—	0	-	—	—	—	
BZ	If $R[SA] = 0$, then $PC \leftarrow PC + 1 + \text{se } IM$	0100000	0	—	01	0	0	00000	1	0	1	Z
BNZ	If $R[SA] \neq 0$, then $PC \leftarrow PC + 1 + \text{se } IM$	1100000	0	—	01	1	0	00000	1	0	1	/Z
JMP	$PC \leftarrow PC + 1 + \text{se } IM$	1000100	0	—	11	—	0	—	1	—	1	
JML	$PC \leftarrow PC + 1 + \text{se } IM, R[DR] \leftarrow PC + 1$	0000111	1	00	11	—	0	00111	1	1	1	PC pass thru

Table 10-20 Control Words for Instructions

```
26      always @(*) begin
27          case (opcode)
28              7'b0000000: control_word = 15'b0_00_00_0_0_00000_0_0_0; // NOP
29              7'b0000010: control_word = 15'b1_00_00_0_0_00010_0_0_0; // ADD
30              7'b0000101: control_word = 15'b1_00_00_0_0_00101_0_0_0; // SUB
31              7'b1100101: control_word = 15'b1_10_00_0_0_00101_0_0_0; // SLT
32              7'b0001000: control_word = 15'b1_00_00_0_0_01000_0_0_0; // AND
33              7'b0001010: control_word = 15'b1_00_00_0_0_01010_0_0_0; // OR
34              7'b0001100: control_word = 15'b1_00_00_0_0_01100_0_0_0; // XOR
35              7'b0000001: control_word = 15'b0_00_00_0_1_00000_0_0_0; // ST
36              7'b0100001: control_word = 15'b1_01_00_0_0_00000_0_0_0; // LD
37              7'b0100010: control_word = 15'b1_00_00_0_0_00010_1_0_1; // ADI
38              7'b0100101: control_word = 15'b1_00_00_0_0_00101_1_0_1; // SBI
39              7'b0101110: control_word = 15'b1_00_00_0_0_01110_0_0_0; // NOT
40              7'b0101000: control_word = 15'b1_00_00_0_0_01000_1_0_0; // ANI
41              7'b0101010: control_word = 15'b1_00_00_0_0_01010_1_0_0; // ORI
42              7'b0101100: control_word = 15'b1_00_00_0_0_01100_1_0_0; // XRI
43              7'b1100010: control_word = 15'b1_00_00_0_0_00010_1_0_0; // AIU
44              7'b1100101: control_word = 15'b1_00_00_0_0_00101_1_0_0; // SIU
45              7'b1000000: control_word = 15'b1_00_00_0_0_00000_0_0_0; // MOV
46              7'b0110000: control_word = 15'b1_00_00_0_0_10000_0_0_0; // LSL
47              7'b0110001: control_word = 15'b1_00_00_0_0_10001_0_0_0; // LSR
48              7'b1100001: control_word = 15'b0_00_10_0_0_00000_0_0_0; // JMR
49              7'b0100000: control_word = 15'b0_00_01_0_0_00000_1_0_1; // BZ
50              7'b1100000: control_word = 15'b0_00_01_1_0_00000_1_0_1; // BNZ
51              7'b1000100: control_word = 15'b0_00_11_0_0_00000_1_0_1; // JMP
52              7'b0000111: control_word = 15'b1_00_11_0_0_00111_1_1_1; // JML
53              default: control_word = 15'bx;
54          endcase
55      end
56  endmodule
```

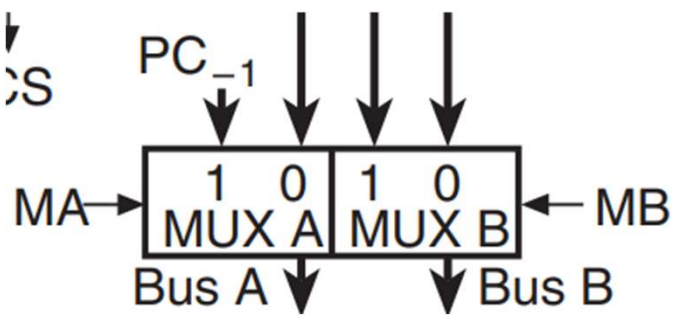
Register File





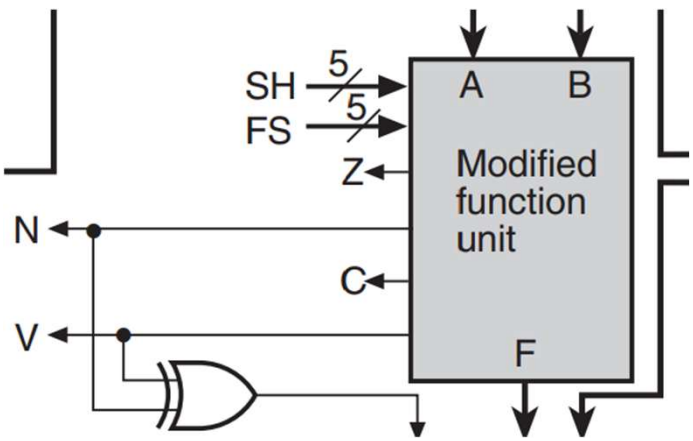

```
module regfile(
    input clk,
    input reset,
    input RW,
    input [4:0] AA,
    input [4:0] BA,
    input [4:0] DA,
    input [31:0] D_data,
    output [31:0] A_data,
    output [31:0] B_data
    );

reg [31:0] data [31:0];
reg [31:0] R5;

assign A_data = data [AA];
assign B_data = data [BA];

integer counter;

initial begin
    for(counter = 0; counter < 32; counter = counter + 1)
        data [counter] = counter;                    // for easier initial debugging
    R5 = data[0];
end

always @(posedge clk)
    begin
        if(reset)
            for(counter = 0; counter < 32; counter = counter + 1)
                data [counter] <= 0;                 // clear all regs on reset
        else if(RW)                                  // if not clearing, then write
            data[DA] <= D_data;
    end

endmodule
```

Mux A and Mux B




```
module mux_A_B (
    input MA, MB,
    input [31:0] PC_1, A_data, IM, B_data,
    output [31:0] bus_A, bus_B
);

    assign bus_A = MA ? PC_1 : A_data;
    assign bus_B = MB ? IM : B_data;

endmodule
```

Function Unit



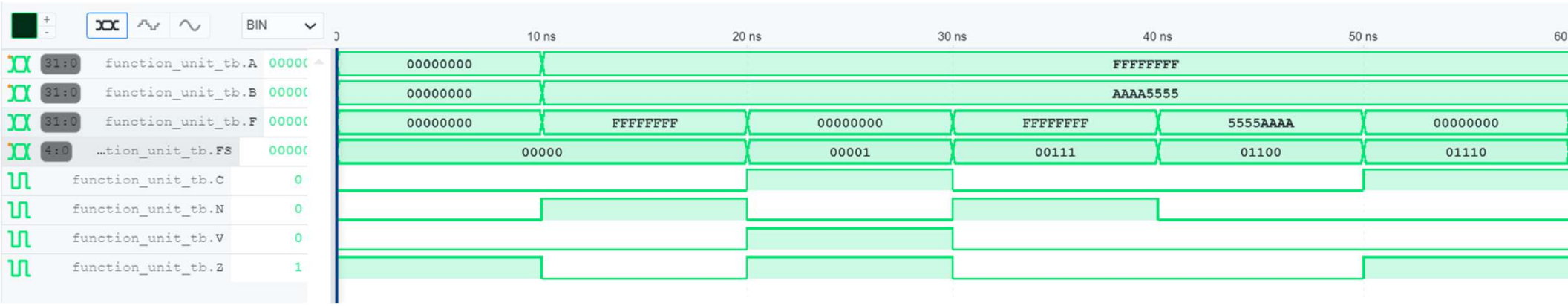
function_unit.v

```
module function_unit (
    input [31:0] A, B,
    input [4:0] FS, SH,
    output reg C, V,
    output Z, N,
    output reg [31:0] F
);
    assign Z = ~|F;
    assign N = F[31];

    always @(*) begin
        case (FS)
            5'b00000: {C, F} = A;
            5'b00001: {C, F} = A + 1;
            5'b00010: {C, F} = A + B;
            5'b00011: {C, F} = A + B + 1;
            5'b00100: {C, F} = A + (~B);
            5'b00101: {C, F} = A - B;
            5'b00110: {C, F} = A - 1;
            5'b00111: {C, F} = A;
            5'b01000: {C, F} = A & B;
            5'b01010: {C, F} = A | B;
            5'b01100: {C, F} = A ^ B;
            5'b01110: {C, F} = ~A;
            5'b10000: {C, F} = A << SH;
            5'b10001: {C, F} = A >> SH;
            default: {C, F} = 0;
        endcase

        V = ( (FS == 5'b00001 | FS == 5'b00010 ) & ( (~A[31] & ~B[31] & F[31]) | (A[31] & B[31] & ~F[31])) ) |
            ( (FS == 5'b00101 | FS == 5'b00110 ) & ( (~A[31] & B[31] & F[31]) | (A[31] & ~B[31] & ~F[31])) ) ;
    end
endmodule
```

Function Unit Testbench

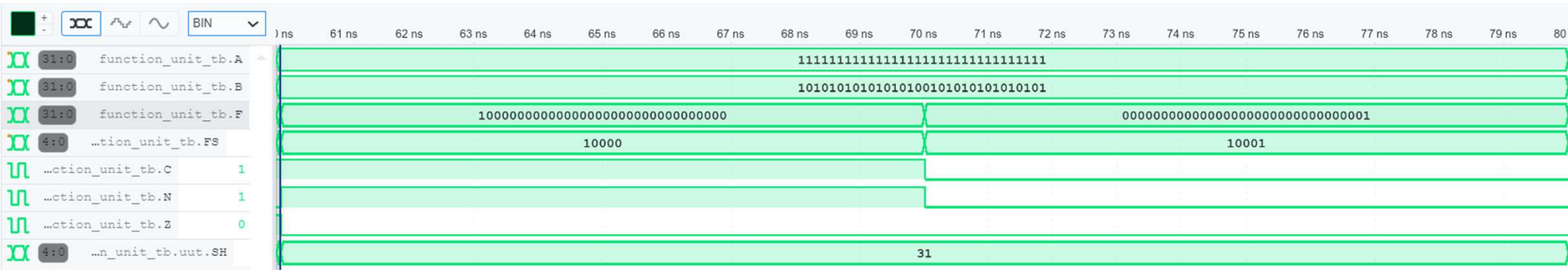
```
`timescale 1ns/1ps
`include "function_unit.v"

module function_unit_tb;
    reg [31:0] A, B;
    reg [4:0] FS, SH;

    wire [31:0] F;
    wire C, V, N, Z;

    function_unit uut(
        .A(A),
        .B(B),
        .FS(FS),
        .SH(SH),
        .F(F),
        .C(C),
        .V(V),
        .N(N),
        .Z(Z)
    );

    integer i;

    initial begin

        $dumpfile("funtion_unit_tb.vcd");
        $dumpvars(0, function_unit_tb);

        {A, B, FS, SH} = 0;
        #10
        A = 32'hFFFF_FFFF;
        B = 32'hAAAA_5555;
        FS = 0; // Transfer A
        #10
        FS = 1; // INC - C, V, Z flags set
        #10
        FS = 7; // Transfer A
        #10
        FS = 12; // XOR - 5555_AAAA expected
        #10
        FS = 14; //NOT - Z flag set
        #10
        SH = 31;
        FS = 16; //LSL - 8000_0000 expected
        #10
        FS = 17; //LSR - 0000_0001 expected
        #10
        {A, B, FS, SH} = 0;
        #10
        A = 32'hAAAA_5555;
        B = 32'hAAAA_5555;
        FS = 2; // ADD - 5554_AAAA expected, C, V flags set
        #10
        FS = 3; // ADD with Carry
        #10
        FS = 4; // A + (~B) - FFFF_FFFF expected
        #10
        FS = 5; // SUB - 0000_0000 expected, Z flag set
        #10
        FS = 6; //DEC - AAAA_5554 expected
        #10
        {A, B, FS, SH} = 0;
        #10
        A = 32'h5555_AAAA;
        B = 32'hAAAA_5555;
        FS = 8; // AND - 0000_0000 expected, Z flag set
        #10
        FS = 10; // OR - FFFF_FFFF expected
        #20;

        $finish;
    end
endmodule
```

Testbench Results



Function	Code
$F = A$	00000
$F = A + 1$	00001
$F = A$	00111
$F = A \oplus B$	01100
$F = \overline{A}$	01110

Testbench Results



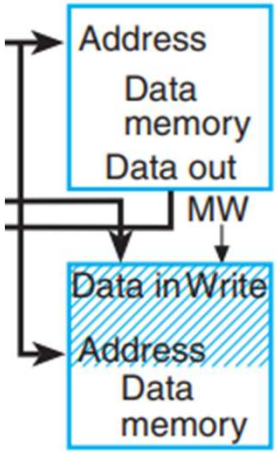
F = sl A 10000

F = sr A 10001

Testbench Results

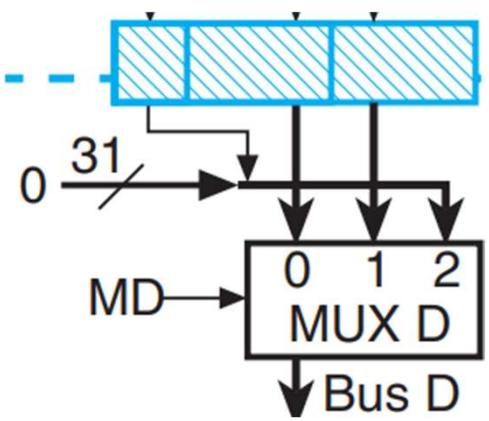
$F = A + B$	00010
$F = A + B + 1$	00011
$F = A + \overline{B}$	00100
$F = A + \overline{B} + 1$	00101
$F = A - 1$	00110
$F = A \wedge B$	01000
$F = A \vee B$	01010

Data Memory




```
module data_memory(
    input clk,
    input [13:0] addr,
    input MW,
    input [31:0] data_in,
    output [31:0] data_out
    );

    reg [31:0] memword [16383:0];   // 16K words (64 KB total)
    integer i;
    wire [31:0] mem_5, mem_6, mem_10;    // To view in the simulation
    assign mem_5 = memword[5];
    assign mem_6 = memword[6];
    assign mem_10 = memword[10];

    initial begin
        for(i=0; i<16384; i=i+1)
            memword[i] <= 0;                    // for easier debugging
    end

    always @ (posedge clk) begin
        if(MW)
            memword[addr] <= data_in;
    end

    assign data_out = memword[addr];

    endmodule
```

Mux D




```
module mux_D (
    input [1:0] MD,
    input [31:0] F, data_out, V_xor_N,
    output reg [31:0] bus_D
);

    always @(*) begin
        case (MD)
            0: bus_D = F;
            1: bus_D = data_out;
            2: bus_D = V_xor_N;
            default: bus_D = 32'bx;
        endcase
    end


endmodule
```

Parameterization

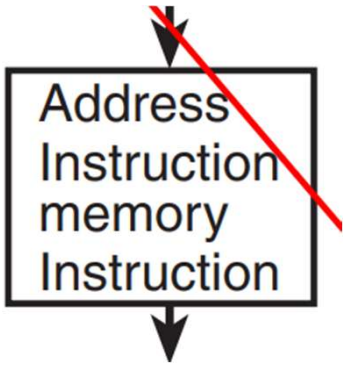
- Opcodes and registers are parameterized to use in the instruction memory module.

asm_param.inc

```
parameter [6:0]  NOP = 7'b0000000;
parameter [6:0]  ADD = 7'b0000010;
parameter [6:0]  SUB = 7'b0000101;
parameter [6:0]  SLT = 7'b1100101;
parameter [6:0]  AND = 7'b0001000;
parameter [6:0]  OR =  7'b0001010;
parameter [6:0]  XOR = 7'b0001100;
parameter [6:0]  ST =  7'b0000001;
parameter [6:0]  LD =  7'b0100001;
parameter [6:0]  ADI = 7'b0100010;
parameter [6:0]  SBI = 7'b0100101;
parameter [6:0]  NOT = 7'b0101110;
parameter [6:0]  ANI = 7'b0101000;
parameter [6:0]  ORI = 7'b0101010;
parameter [6:0]  XRI = 7'b0101100;
parameter [6:0]  AIU = 7'b1100010;
parameter [6:0]  SIU = 7'b1000101;
parameter [6:0]  MOV = 7'b1000000;
parameter [6:0]  LSL = 7'b0110000;
parameter [6:0]  LSR = 7'b0110001;
parameter [6:0]  JMR = 7'b1100001;
parameter [6:0]  BZ =  7'b0100000;
parameter [6:0]  BNZ = 7'b1100000;
parameter [6:0]  JMP = 7'b1000100;
parameter [6:0]  JML = 7'b0000111;
```

```
parameter [4:0]  R0  = 5'd0;
parameter [4:0]  R1  = 5'd1;
parameter [4:0]  R2  = 5'd2;
parameter [4:0]  R3  = 5'd3;
parameter [4:0]  R4  = 5'd4;
parameter [4:0]  R5  = 5'd5;
parameter [4:0]  R6  = 5'd6;
parameter [4:0]  R7  = 5'd7;
parameter [4:0]  R8  = 5'd8;
parameter [4:0]  R9  = 5'd9;
parameter [4:0]  R10 = 5'd10;
parameter [4:0]  R11 = 5'd11;
parameter [4:0]  R12 = 5'd12;
parameter [4:0]  R13 = 5'd13;
parameter [4:0]  R14 = 5'd14;
parameter [4:0]  R15 = 5'd15;
parameter [4:0]  R16 = 5'd16;
parameter [4:0]  R17 = 5'd17;
parameter [4:0]  R18 = 5'd18;
parameter [4:0]  R19 = 5'd19;
parameter [4:0]  R20 = 5'd20;
parameter [4:0]  R21 = 5'd21;
parameter [4:0] R22 = 5'd22;
parameter [4:0] R23 = 5'd23;
parameter [4:0] R24 = 5'd24;
parameter [4:0] R25 = 5'd25;
parameter [4:0] R26 = 5'd26;
parameter [4:0] R27 = 5'd27;
parameter [4:0] R28 = 5'd28;
parameter [4:0] R29 = 5'd29;
parameter [4:0] R30 = 5'd30;
parameter [4:0] R31 = 5'd31;
```

Instruction Memory




```
module instruction_mem (
    input [13:0] PC,
    output [31:0] IR
);
    `include "asm_param.inc"

    reg [31:0] memword [16383:0];   // 16K words (64 KB total)
    assign IR = memword[PC];

    integer i;

    task insert_nops;
        input integer addr;  // Address where NOPs should be inserted
    begin
        memword[addr]   = {NOP, R0, R0, 15'd0};
        memword[addr+1] = {NOP, R0, R0, 15'd0};
        memword[addr+2] = {NOP, R0, R0, 15'd0};
    end
endtask
```

Instruction Memory

```
    initial begin
        memword[0] = {ADI, R5, R5, 15'd10};
        insert_nops(1);
        memword[4] = {ADI, R6, R6, 15'd15};
        insert_nops(5);
        memword[8] = {ADD, R7, R5, R6, 10'd0};
        insert_nops(9);
        memword[12] = {MOV, R10, R7, 15'd0};
        insert_nops(13);
        memword[16] = {ST, R5, R5, R10, 10'd0};
        insert_nops(17);
        memword[20] = {LD, R11, R5, 15'd0};
        insert_nops(21);
        memword[24] = {SUB, R12, R5, R11, 10'd0};   // R5 - R11 = 10 - 25
        insert_nops(25);
        memword[28] = {SLT, R13, R5, R10, 10'd0}; // R13 = 1, if R5 < R10
        insert_nops(29);
        memword[32] = {ADI, R8, R8, 15'd5};     // 5 in R8
        insert_nops(33);
        memword[36] = {AND, R9, R5, R8, 10'd0}; // 5 & 10 = 0
        insert_nops(37);
        memword[40] = {OR, R10, R5, R8, 10'd0}; // 5 | 10 = 15 : R14 = 15
        insert_nops(41);
        memword[44] = {XOR, R12, R5, R8, 10'd0}; // 5 ^ 10 = 15: R15 = 15
        insert_nops(45);
        memword[48] = {SBI, R10, R11, 15'd3}; // R16 = R11 - 3 = 25 - 3 = 22
        insert_nops(49);
        memword[52] = {NOT, R10, R5, 15'd30};
        insert_nops(53);
        memword[54] = {ANI, R10, R5, 15'd12}; //
        insert_nops(55);
        memword[58] = {ORI, R10, R5, 15'd10}; // R10 = 15
        insert_nops(59);
        memword[62] = {XRI, R10, R5, 15'd15}; // R10 = 5
        insert_nops(63);
        memword[66] = {AIU, R10, R5, 15'o77771}; // R10 = 32767 + 5
        insert_nops(67);
        memword[70] = {SIU, R10, R5, 15'o77771}; // R10 = 5
        insert_nops(71);
        memword[74] = {ADI, R5, R5, 15'd30};
        insert_nops(75);
        memword[78] = {JMP, R0, R0, 15'o77771}; // -1 in 15 bit octadecimal - Jump 1 line up


        for (i = 79; i < 16384; i = i + 1) begin
            memword[i] = {NOP, R0, R0, 15'd0};  //32'd0
        end
    end
endmodule
```

Top Module

```verilog
`include "instruction_mem.v"
`include "instruction_decoder.v"
`include "constant_unit.v"
`include "regfile.v"
`include "mux_A_B.v"
`include "function_unit.v"
`include "data_memory.v"
`include "mux_C.v"
`include "mux_D.v"

module risc_cpu (
    input clk, reset
);

    // Registers to view in the simulation.
    wire [31:0] R0;
    wire [31:0] R1;
    wire [31:0] R2;
    wire [31:0] R3;
    wire [31:0] R4;
    wire [31:0] R5;
    wire [31:0] R6;
    wire [31:0] R7;
    wire [31:0] R8;
    wire [31:0] R9;
    wire [31:0] R10;
    wire [31:0] R11;
    wire [31:0] R12;
    wire [31:0] R13;
    wire [31:0] R14;
    wire [31:0] R15;
    wire [31:0] R16;
    wire [31:0] R17;
    wire [31:0] R18;
    wire [31:0] R19;
    wire [31:0] R20;
    wire [31:0] R21;
    wire [31:0] R22;
    wire [31:0] R23;
    wire [31:0] R24;
    wire [31:0] R25;
    wire [31:0] R26;
    wire [31:0] R27;
    wire [31:0] R28;
    wire [31:0] R29;
    wire [31:0] R30;
    wire [31:0] R31;
```

```verilog
    assign R0 = registers.data[0];
    assign R1 = registers.data[1];
    assign R2 = registers.data[2];
    assign R3 = registers.data[3];
    assign R4 = registers.data[4];
    assign R5 = registers.data[5];
    assign R6 = registers.data[6];
    assign R7 = registers.data[7];
    assign R8 = registers.data[8];
    assign R9 = registers.data[9];
    assign R10 = registers.data[10];
    assign R11 = registers.data[11];
    assign R12 = registers.data[12];
    assign R13 = registers.data[13];
    assign R14 = registers.data[14];
    assign R15 = registers.data[15];
    assign R16 = registers.data[16];
    assign R17 = registers.data[17];
    assign R18 = registers.data[18];
    assign R19 = registers.data[19];
    assign R20 = registers.data[20];
    assign R21 = registers.data[21];
    assign R22 = registers.data[22];
    assign R23 = registers.data[23];
    assign R24 = registers.data[24];
    assign R25 = registers.data[25];
    assign R26 = registers.data[26];
    assign R27 = registers.data[27];
    assign R28 = registers.data[28];
    assign R29 = registers.data[29];
    assign R30 = registers.data[30];
    assign R31 = registers.data[31];
```

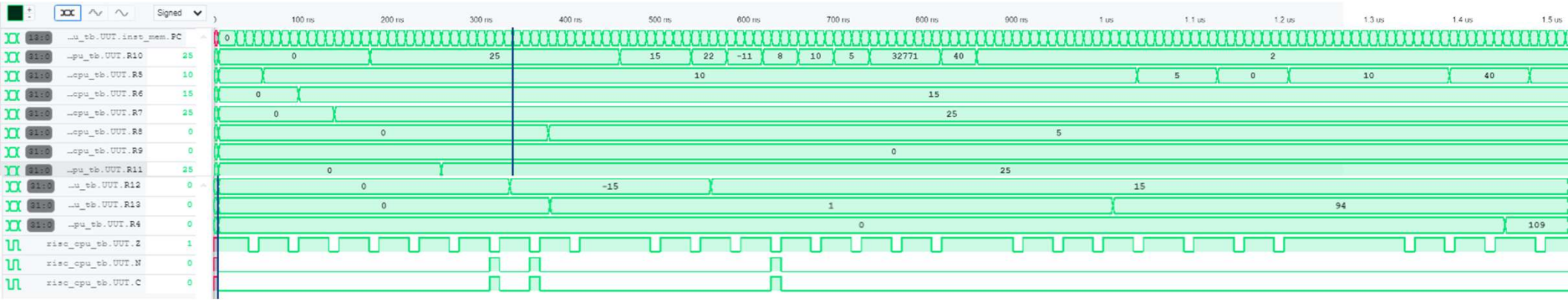
```verilog
    // Variables and module instantiations for each stage

    // IF
    reg [31:0] PC, PC_1, IR;
    wire [31:0] instruction, PC_incremented;

    assign PC_incremented = PC + 1;

    instruction_mem inst_mem(
        .PC(PC[13:0]),
        .IR(instruction)
    );

    // DOF
    reg [31:0] PC_2, bus_A, bus_B;
    reg RW_DOF, PS, MW;
    reg [1:0] MD_DOF, BS;
    reg [4:0] DA_DOF, FS, SH;
    wire RW_wire, PS_wire, MW_wire, MA_wire, MB_wire, CS_wire;
    wire [1:0] MD_wire, BS_wire;
    wire [4:0] DA_wire, FS_wire, SH_wire, AA_wire, BA_wire;
    wire [14:0] IM;
    wire [31:0] IM_extended, A_data, B_data, bus_A_wire, bus_B_wire;

    assign IM = IR[14:0];
    assign SH_wire = IR[4:0];

    instruction_decoder inst_dec(
        .IR(IR),
        .RW(RW_wire), .MW(MW_wire), .MB(MB_wire), .MA(MA_wire), .CS(CS_wire), .PS(PS_wire),
        .MD(MD_wire), .BS(BS_wire),
        .FS(FS_wire), .AA(AA_wire), .BA(BA_wire), .DA(DA_wire)
    );

    constant_unit const_unit(
        .CS(CS_wire), .IM(IM), .IM_extended(IM_extended)
    );

    regfile registers(
        .clk(clk), .reset(reset), .RW(RW),
        .AA(AA_wire), .BA(BA_wire), .DA(DA), .D_data(bus_D_wire),
        .A_data(A_data), .B_data(B_data)
    );
```

```
    mux_A_B muxAB(
        .MA(MA_wire), .MB(MB_wire),
        .PC_1(PC_1), .A_data(A_data), .B_data(B_data), .IM(IM_extended),
        .bus_A(bus_A_wire), .bus_B(bus_B_wire)
    );

    // EX
    reg RW, V_xor_N;
    reg [4:0] DA;
    reg [1:0] MD;
    reg [31:0] F, data_out;

    wire C, V, N, Z, V_xor_N_wire;
    wire [1:0] mux_C_sel_wire;
    wire [31:0] BrA, F_wire, data_out_wire, PC_wire;

    assign BrA = PC_2 + bus_B;
    assign mux_C_sel_wire = {BS[1], ( ((PS^Z) | BS[1]) & BS[0])};
    assign V_xor_N_wire = V^N;

    function_unit func_unit(
        .A(bus_A), .B(bus_B), .FS(FS), .SH(SH),
        .C(C), .V(V), .Z(Z), .N(N), .F(F_wire)
    );

    data_memory data_mem(
        .clk(clk), .addr(bus_A[13:0]), .MW(MW), .data_in(bus_B),
        .data_out(data_out_wire)
    );

    mux_C muxC(
        .sel(mux_C_sel_wire), .PC_1(PC_incremented), .BrA(BrA), .RAA(bus_A),
        .PC(PC_wire)
    );

    // WB
    wire [31:0] bus_D_wire;
    mux_D muxD(
        .MD(MD), .F(F), .data_out(data_out), .V_xor_N({31'b0, V_xor_N}),
        .bus_D(bus_D_wire)
    );

    always @(posedge clk) begin
        if (reset) begin
            {PC, PC_1, IR, PC_2, RW_DOF, DA_DOF, MD_DOF, BS, PS, MW,
            FS, SH, bus_A, bus_B, RW, DA, MD, V_xor_N, F, data_out} <= 0;
        end

        else begin
            // IF Stage
            PC_1 <= PC_incremented;
            IR <= instruction;

            // DOF Stage
            PC_2 <= PC_1;
            RW_DOF <= RW_wire;
            DA_DOF <= DA_wire;
            MD_DOF <= MD_wire;
            BS <= BS_wire;
            PS <= PS_wire;
            MW <= MW_wire;
            FS <= FS_wire;
            SH <= SH_wire;
            bus_A <= bus_A_wire;
            bus_B <= bus_B_wire;

            // EX Stage
            RW <= RW_DOF;
            DA <= DA_DOF;
            MD <= MD_DOF;
            PC <= PC_wire;

            V_xor_N <= V_xor_N_wire;
            F <= F_wire;
            data_out <= data_out_wire;

        end
    end
endmodule
```

Testbench

```
`timescale 1ns/1ps
`include "risc_cpu.v"

module risc_cpu_tb;
    reg clk;
    reg reset;

    risc_cpu UUT(
        .clk(clk),
        .reset(reset)
    );

    always #5 clk = ~clk;

    integer i;

    initial begin
        $dumpfile("risc_cpu_tb.vcd");
        $dumpvars(0, risc_cpu_tb);

        clk = 0;
        reset = 1;
        #20

        reset = 0;

        #1000

        $finish;
    end
endmodule
```

Testbench Results



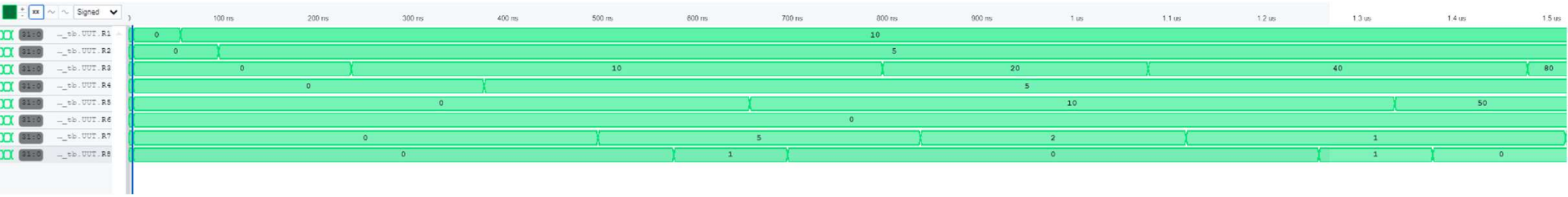
Instructions for Multiplication

```
module instruction_mem (
    input [13:0] PC,
    output [31:0] IR
);
    `include "asm_param.inc"

    reg [31:0] memword [16383:0];   // 16K words (64 KB total)
    assign IR = memword[PC];

    integer i;

    task insert_nops;
        input integer addr;  // Address where NOPs should be inserted
    begin
        memword[addr]   = {NOP, R0, R0, 15'd0};
        memword[addr+1] = {NOP, R0, R0, 15'd0};
        memword[addr+2] = {NOP, R0, R0, 15'd0};
    end
endtask

    initial begin
        // Store number A into R1
        memword[0] = {ADI, R1, R1, 15'd10};  // A = 10
        insert_nops(1);

        // Store number B into R2
        memword[4] = {ADI, R2, R2, 15'd5};   // B = 5
        insert_nops(5);

        // Check and store absolute values
        memword[8]  = {SLT, R8, R1, R0, 10'd0}; // If A < 0, R8 = 1
        insert_nops(9);
        memword[12] = {BZ, R0, R8, 15'd8};     // If A >= 0, skip negation (jump to POS_A) 12+1+8=21
        insert_nops(13);
        memword[16] = {SUB, R3, R0, R1, 10'd0}; // R3 = -A (absolute value of A)
        insert_nops(17);
        memword[20] = {JMP, R0, R0, 15'd4};    // Jump to AFTER_A (address 28)
        insert_nops(21);

        // POS_A: If A >= 0, use R1 as is (positive A)
        memword[24] = {MOV, R3, R1, 15'd0};     // R3 = A (already positive)
        insert_nops(25);

        // AFTER_A: Check if B is negative
        memword[28] = {SLT, R8, R2, R0, 10'd0}; // If B < 0, R8 = 1
        insert_nops(29);
        memword[32] = {BZ, R0, R8, 15'd8};    // If B >= 0, skip negation (jump to POS_B)
        insert_nops(33);
        memword[36] = {SUB, R4, R0, R2, 10'd0}; // R4 = -B (absolute value of B)
        insert_nops(37);
        memword[40] = {JMP, R0, R0, 15'd4};    // Jump to AFTER_B (address 48)
        insert_nops(41);

        // POS_B: If B >= 0, use R2 as is (positive B)
        memword[44] = {MOV, R4, R2, 15'd0};     // R4 = B (already positive)
        insert_nops(45);

        // AFTER_B: Initialize result registers (R5 for lower 32 bits, R6 for upper 32 bits)
        memword[48] = {MOV, R5, R0, 15'd0};     // Lower 32 bits = 0
        insert_nops(49);
        memword[52] = {MOV, R6, R0, 15'd0};     // Upper 32 bits = 0
        insert_nops(53);
        memword[56] = {MOV, R7, R4, 15'd0};     // Loop counter = B (R4)
        insert_nops(57);

        // LOOP: Multiplication loop
        memword[60] = {BZ, R0, R7, 15'd36};      // If counter is 0, exit loop. Jump to DONE 60+1+36=97
        insert_nops(61);
```

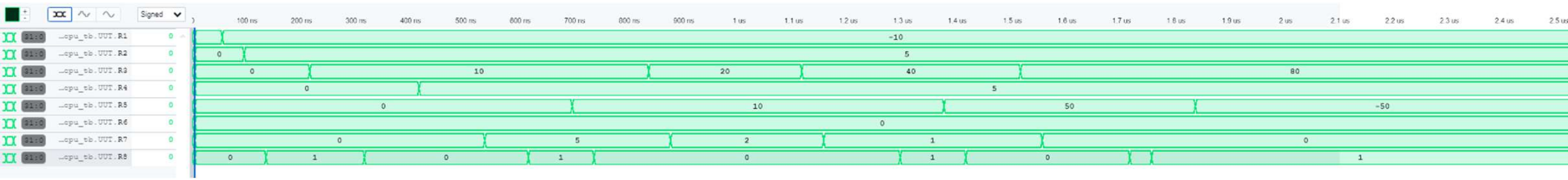
```
        memword[64] = {ANI, R8, R7, 15'd1};     // Check if LSB of counter is
        insert_nops(65);
        memword[68] = {BZ, R0, R8, 15'd16};     // If not, skip addition (jur
        insert_nops(69);
        memword[72] = {ADD, R5, R5, R3, 10'd0}; // Add R3 (A) to result lower
        insert_nops(73);

        // Handle carry
        memword[76] = {SLT, R8, R5, R3, 10'd0};      // If R5 < R3 after add:
        insert_nops(77);
        memword[80] = {BZ, R0, R8, 15'd4};          // If no carry, skip incr
        insert_nops(81);
        memword[84] = {ADI, R6, R6, 15'd1};      // Increment upper 32-bit pa
        insert_nops(85);

        // NO_CARRY: SKIP_ADD
        memword[88] = {LSL, R3, R3, 15'd1};           // Left shift A (R3)
        insert_nops(89);
        memword[92] = {LSR, R7, R7, 15'd1};          // Right shift counter
        insert_nops(93);
        memword[96] = {JMP, R0, R0, 15'o77730};         // JMP LOOP 96+1-40=5
        insert_nops(97);

        // DONE:
        memword[100] = {XOR, R8, R1, R2, 10'd0};     // XOR R8, R1, R2
        insert_nops(101);
        memword[104] = {LSR, R8, R8, 15'd31};            // LSR R8, R8, 31
        insert_nops(105);
        memword[108] = {BZ, R0, R8, 15'd8};        // BZ END              //
        insert_nops(109);
        memword[112] = {SUB, R5, R0, R5, 10'd0};     // SUB R5, R0, R5
        insert_nops(113);
        memword[116] = {SUB, R6, R0, R6, 10'd0};     // SUB R6, R0, R6      // Negate upper 32 bits

        // END:
        memword[117] = {NOP, R0, R0, 15'd0};         // Final NOP

        for (i = 118; i < 16384; i = i + 1) begin
            memword[i] = {NOP, R0, R0, 15'd0};  //32'd0
        end
    end
endmodule
```

- Multiplication results are stored in R5 (lower 32 bits) and R6 (upper 32 bits)

Multiplying 10 and 5

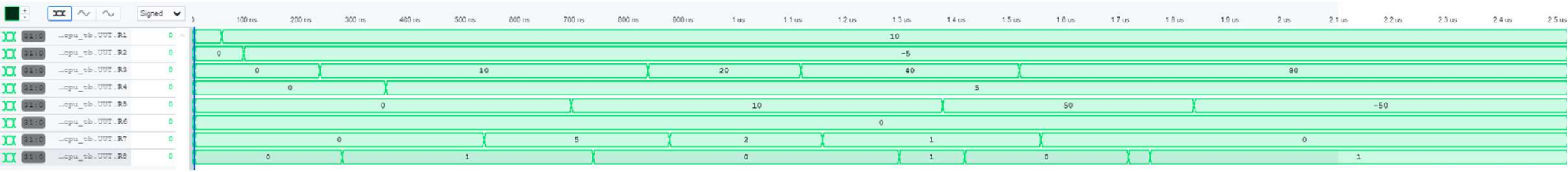
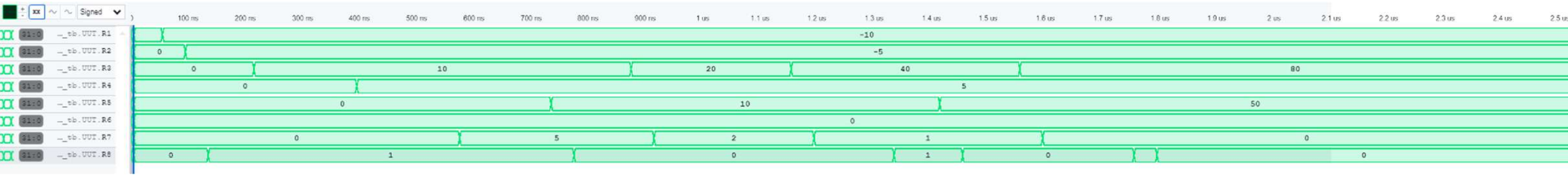


Multiplying -10 and 5



- Multiplication results are stored in R5 (lower 32 bits) and R6 (upper 32 bits)

Multiplying 10 and -5



Multiplying -10 and -5